💬 Lost Signal — Real-Time Narrative Messaging Engine (Godot 4.6)

> Build interactive story games through a real-time messaging interface — the kind players recognize from Discord, Teams, and modern chat apps. **No programming required. Stories live in JSON files.**

🎮 What is Lost Signal?

Lost Signal is a narrative engine for Godot 4 that lets you build interactive story-driven games where everything unfolds through a **real-time messaging interface**.

Characters talk to the player through a desktop chat system:

- multiple simultaneous conversations
- branching dialogue with conditions and variables
- media messages (images, audio clips)
- background events that trigger while the player reads

You don't write code. You don't modify the engine. You write **JSON story files** — structured like templates, readable like scripts.

🖥️ A desktop-first narrative experience

Unlike mobile texting games, Lost Signal is designed for **PC storytelling** inspired by modern communication tools — Discord, Slack, Teams.

This unlocks narrative structures that don't exist in traditional visual novels:

- parallel conversations unfolding independently
- background characters sending messages mid-scene
- interruptions and notification-driven pacing
- multi-threaded storytelling across contacts

✨ What you can create

💔 Relationship-driven stories

Friendships, breakups, emotional dialogue, character-driven drama.

🧠 Psychological & experimental fiction

Unreliable messages, shifting perspectives, fragmented communication.

🕵️ Thriller & mystery

Multi-character investigations, conflicting narratives, hidden information.

🚀 Sci-fi communication stories

Remote survival messaging, system-based storytelling (*Lifeline*-style).

😨 Horror

Corrupted messages, disappearing contacts, unstable conversations.

⚡ Key Features

💬 Real-time messaging system

- Discord-style chat interface, built for PC
- Multiple active conversations with contact switching
- Animated typing indicator (three dots)
- Timestamps and natural message flow
- Contact status indicators: online, away, offline, network issue — each a narrative tool, not just a UI state. A contact with *network issue* that keeps dropping in and out tells a story before a single message is sent.

🧠 Full narrative logic — no scripting

Write your entire story in JSON. The engine handles the rest.

- **Branching dialogue** — choices that send player messages and advance the story. A choice can send a single message or a sequence of bubbles in a row — the way people actually text.
- **Flags** — boolean states for simple branching
- **Variables** — numeric values for complex systems (trust, stress, relationship scores…)
- **Conditions** — show messages or choices based on flags and variables, with full `and` / `or` / nested logic
- **Effects** — modify variables, change contact status, rename contacts mid-story. A contact listed as "Unknown Number" can reveal their name at the exact moment the story calls for it.
- **Templates** — inject variable values into message text: `"Thanks {player_name}."`
- **Free input** — ask the player to type a free-text response and store it as a variable

---

🔀 Background conversations

Characters don't wait for the player to finish reading.

Scenes can be set to **trigger automatically** after another scene ends, or **resume after a specific story flag** is set. This means a second character can message the player mid-conversation — the notification badge appears, and the player decides when to switch.

Background scenes run independently, are saved automatically, and persist across sessions.

🖍️ Live message editing

Characters can modify their own messages after sending — correcting a typo with a delay, or deleting a message entirely. A mechanic that doesn't exist in traditional dialogue systems, and one that opens specific narrative possibilities: hesitation, regret, second thoughts, unreliable communication.

📎 Media messaging

- 📷 Images sent as chat bubbles, tappable to view fullscreen
- 🔊 Audio messages with a dedicated playback UI (progress bar, duration)
- Both are fully integrated into the save system

🌍 Built-in localization

Lost Signal is built for multilingual projects from the start.

- Dialogue files are selected per language: `scene.fr.json`, `scene.en.json`
- The engine picks the right file automatically based on the active language
- Falls back to the base file if no translation exists yet
- All UI text (buttons, status labels, menus) is translated via a single CSV file — adding a new language takes minutes
- Language is auto-detected from the player's system on first launch
- Players can change language in-game at any time without losing progress

---

🔍 Built-in story validator

Write with confidence — the engine checks your work before the player sees it.

On every launch, Lost Signal validates all your JSON files and reports:

- missing or broken scene references
- undefined flags used in conditions
- malformed effects or conditions
- unreachable choices or dead ends

Errors and warnings appear **directly in the game window** — no console, no external tool, no guesswork.

⚙️ Player settings — out of the box

A fully functional settings menu is included with no setup required:

- **Language** — switches dialogue and UI language instantly
- **Volume** — master volume slider
- **Resolution** — from 480p to 4K
- **Display mode** — Windowed, Fullscreen Windowed (borderless), or Exclusive Fullscreen

Settings persist between sessions and are applied automatically on launch.

🎨 Fully customizable interface

- Theme system via `theme.json` — colors, typography, bubble sizing, typing speed
- No code changes needed to reskin the entire game
- Designed to adapt to different visual identities

💾 Automatic save system

- Story state is saved **after every player choice and every received message** — players never lose progress
- Full persistence: variables, flags, conversation history, contact names and statuses
- Main menu with New Game / Continue
- Save file is human-readable JSON

🛠 Authoring workflow

A complete game requires only:

```
story.json           ← contacts and starting scene
dialogues/*.json     ← your story scenes
assets/              ← images and audio (optional)
theme.json           ← visual style (optional)
```

No Godot editor knowledge required to write content. No scripting, no nodes, no scenes to touch.

🧩 How it works

1. Define your characters in `story.json`
2. Write story scenes in JSON files
3. Drop the files into `/dialogues`
4. Add images or audio if needed
5. Run in Godot — the validator reports any issues immediately

That's it.

📄 Example scene

```
{
  "id": "intro",
  "messages_in": [
    { "text": "Are you there?", "pause": "short" },
    { "text": "We need to talk." }
  ],
  "choices": [
    {
      "text": "Who is this?",
      "message": "Who are you?",
```

```
      "next": "scene_reveal",
      "flag": "asked_identity",
      "effects": [{ "op": "add", "var": "trust", "value": 1 }]
    },
    {
      "text": "Wrong number.",
      "message": "I think you have the wrong number.",
      "next": "scene_denial"
    }
  ]
}
```

Two messages arrive. The player picks a response. A variable updates. The story continues — no code written.

🧪 Perfect for

- Narrative-driven indie games
- Interactive fiction and digital stories
- Game jams — a full prototype in hours
- Character-focused drama and emotional narratives
- Sci-fi, mystery, horror, and experimental storytelling
- Writers and designers with no game development background

📦 What's included

- Full Godot 4.6 project — open and run immediately
- Complete messaging UI (typing indicators, media bubbles, contact panel)
- JSON narrative engine with conditions, variables, flags, effects, templates
- Multi-contact system with status indicators
- Background conversation and trigger system
- Built-in story validator with in-game reporting
- Auto-save system (saves after every story beat)
- Main menu (New Game / Continue)
- Player settings menu (language, volume, resolution, display mode)
- Localization system (per-language dialogue files + UI translation CSV)
- Theme system (`theme.json`)
- Playable demo scenario
- Bilingual authoring guide (EN + FR) with full syntax reference

---

🚀 Why Lost Signal?

Most narrative tools fall into one of three categories:

- **Too technical** — requires programming to do anything meaningful
- **Too limited** — simple linear branching, no variables, no parallel threads
- **Too generic** — not designed around real-time communication as the core mechanic

Lost Signal is built specifically for messaging-based narratives.

The interface is not a skin over a traditional dialogue system. It is the system — with the timing, the interruptions, the multi-threaded pacing that make communication stories feel real.

🧠 Design philosophy

> Communication itself is the narrative system.

Messages are not just dialogue. They are:

- events
- interruptions
- contradictions
- delayed revelations
- evolving information streams

Lost Signal gives you the tools to use all of that.

📋 Before you start

Lost Signal is designed to be accessible to writers and designers — but a few things are worth knowing upfront.

JSON is the authoring format. You don't write code, but you do write structured text files. A basic familiarity with JSON syntax (curly braces, quotes, commas) will save you time. The built-in validator catches most mistakes, and any text editor with JSON highlighting makes the experience smoother.

Linear stories are easy. Parallel stories take planning. A single-contact story with branching choices is straightforward to build. Multi-contact narratives — where characters message you independently and interrupt each other — require thinking through the structure before you write. The tools are there; the design work is yours.

The authoring guide is your reference, not a tutorial. `docs/authoring.md` covers every field and every feature, but it assumes you're looking something up, not learning from scratch. Start with the demo scenario and modify it — that's the fastest way in.

⚠️ Requirements

- Godot 4.6
- No external dependencies or plugins